

Hermes Agent 自进化机制源码阅读报告

围绕 handwritten questions: skill 分类、检索冲突治理、沉淀标准、更新与新增边界

项目	内容
分析对象	Hermes Agent 的 Skills System、skill_manage 写入路径、background_review 自进化审查、Curator 生命周期治理，以及外部 hermes-agent-self-evolution 优化管线。
输出目标	把“自进化”从源码级触发机制、决策准则、数据结构、冲突治理、工程改进方案几个层面解释清楚。
日期	2026-07-09
说明	第 1 题照片末尾字样不完全清晰，本文按“新增 skill 的 category 判断与服务/环境绑定”展开；第 3 题按“自进化沉淀标准与反复 patch 治理”展开。

核心结论

- Hermes 的自进化不是模型权重训练，而是把一次任务中的可复用 workflow、纠错、工具用法、pitfall 和用户偏好沉淀为 Skill 或 Memory。核心写入口是 skill_manage，核心触发点在 turn finalizer 后台 review。
- 新增 skill 的 category 在当前源码中主要是目录组织维度，验证偏语法性；真正的“是否适用某服务/环境”应依赖 frontmatter、environments、platforms、prerequisites、required_environment_variables、tags 和正文触发条件共同表达。
- 源码中的新增/更新优先级很明确：优先更新本轮加载过的 skill，其次更新已有 umbrella skill，再其次给已有 umbrella 增加 support file，最后才创建新的 class-level umbrella skill。
- 规模变大后的主要风险不是能不能写入，而是语义重复、触发条件重叠、互相矛盾、服务绑定错误和陈旧 skill 干扰检索。需要索引层、冲突检测层、质量门禁层和 Curator 生命周期层一起治理。

目录

1. 手写问题转写与整体回答框架
2. Hermes 自进化的系统边界：不是训练权重，而是维护可检索过程记忆
3. 源码级调用链：从一轮对话结束到 skill_manage 写入
4. 问题一：新增 skill 如何判断所属 category，以及是否绑定服务/环境
5. 问题二：规模庞大时如何检索 skill 并避免冲突
6. 问题三：自进化的沉淀标准、评估标准与反复 patch 治理
7. 问题四：进化后如何判断新增 skill 还是改进既有 skill
8. 建议的工程化算法：候选检索、相似度评分、冲突评分、写入决策
9. Hermes 现有实现的边界与可增强点
10. 可直接落地的实现清单
11. 证据来源与源码文件索引

1. 手写问题转写与整体回答框架

根据上传照片，问题大致可以转写为以下四类。个别字迹存在不确定性，本文按 Hermes skill 自进化机制的语义进行了归并。

序号	问题转写	本文回答重点
1	新增 skill 如何判断所属 category，是否需要绑定服务/环境？	解释 category 在源码中的目录含义、frontmatter/环境标签/依赖字段如何承担“绑定”职责，并给出分类决策树。
2	如何检索 skill 与避免冲突，尤其规模庞大时？	解释现有 skills_list/skill_view 和 name collision 检查，再补充面向大规模 skill 库的多级索引、冲突检测和合并策略。
3	自进化沉淀标准、评估标准，以及如何避免反复贴/反复 patch？	解释 background review prompt 中的写入信号、过滤规则、质量门禁和 patch 治理。
4	当进化后如何判断是新增 skill，还是改进既有 skill？标准和边界是什么？	给出 Hermes 当前源码中的优先级：loaded skill -> existing umbrella -> support file -> new class-level umbrella，并转化为决策表。

总的回答框架如下：先说明 Hermes 把 skill 当作“按需加载的过程记忆”；再从源码调用链解释后台 review 的触发和写入；最后分别回答 category、检索冲突、沉淀标准、新增/更新边界四个问题。

```
User turn finished
-> turn_finalizer checks skill review interval
-> background_review fork receives conversation snapshot
-> reviewer decides: no-op / update memory / update skill / create skill
-> skill_manage writes SKILL.md or supporting files
-> skill_usage sidecar records provenance and usage
-> curator later marks stale/archive/consolidates duplicates
```

2. Hermes 自进化的系统边界：不是训练权重，而是维护可检索过程记忆

2.1 Skill 在 Hermes 中是什么

Hermes 的 skill 可以理解作为一种可检索、可按需展开的“过程性知识单元”。它不是简单提示词，也不是模型参数，而是一组目录化文件：核心是 SKILL.md，旁边可以带 references、templates、examples、assets、scripts 等支持文件。官方 Skills System 文档强调 progressive disclosure，即先只暴露列表和描述，只有相关时才查看完整 SKILL.md，需要更细信息时再打开支持文件。[S1][S2]

组成	作用	工程含义
SKILL.md	技能主文档，包含 YAML frontmatter 与正文步骤。	检索首先看 name/description；执行时再看完整步骤。
frontmatter	声明 name、description，以及平台、环境、标签、依赖等元信息。	决定是否容易被检索、是否适当前环境、是否会被误用。
references/	长篇背景、API 说明、细节手册。	避免把主文档写得过长；符合 progressive disclosure。
templates/	模板、配置样例、prompt 模板。	适合可复用但不应污染主流程的材料。
scripts/	可执行辅助脚本。	让 skill 不只是文字，还能封装稳定工具链。
.usage.json	usage sidecar，记录 view/use/patch_count、created_by、lifecycle、pinned 等。	为 Curator 的 stale/archive/consolidation 提供依据。

2.2 自进化在 Hermes 里的准确含义

Hermes 的“自进化”至少有两层含义。第一层是运行时自学习：一轮任务结束后，后台 reviewer 从对话快照里判断是否有可复用经验，必要时用 skill_manage 写入或修改 skill。第二层是外部优化管线：hermes-agent-self-evolution 项目使用 DSPy/GEPA 等方法优化 skill 文件、tool description、system prompt，甚至代码，但它是“作用于 Hermes 的离线优化器”，不是 Hermes 主进程内部的权重训练。[S3][S4]

层次	触发位置	改什么	是否训练模型权重	风险控制
运行时 skill 自学习	对话完成后的 background_review fork	Memory、Skill 文件、支持文件	否	tool whitelist、read-before-write、protected skill guard、security scan、用户审批/写入门禁
Curator 生命周期治理	后台定时维护或手动 curator run	active/stale/archived 状态、重复 skill 合并、报告和备份	否	pin、hub/bundled 保护、backup/rollback
外部 self-evolution pipeline	独立命令行或 CI 流水线	skill 文件、工具描述、提示词、代码候选变体	否	eval、semantic preservation、tests、size limit、PR review

2.3 为什么它不像普通 prompt 库

- 普通 prompt 库通常一次性塞上下文，容易污染窗口；Hermes skill 采用先列表、再查看、再打开支持文件的分级加载。
- 普通经验库缺少生命周期；Hermes 通过 .usage.json 和 Curator 记录使用、修补、归档和合并。
- 普通笔记难以执行；Hermes skill 可以携带模板、脚本和资源文件，使经验成为可执行工作流。
- 普通自动写入容易越权；Hermes 的 background review 使用工具白名单，且 skill_manage 内部还有 read-before-write、保护目录、security scan 等门禁。

3. 源码级调用链：从一轮对话结束到 skill_manage 写入

3.1 触发点：turn_finalizer 的间隔判断

在 Hermes 运行时，自进化 review 不是每次用户输入都同步阻塞主回答，而是在一轮 turn 结束后由 finalizer 判断是否需要后台 fork。源码中的关键条件可以概括为：存在 skill nudge interval，距离上次 skill review 的工具调用/迭代数达到阈值，并且 skill_manage 工具可用。触发后会重置计数并在主回答之后启动后台 review。[S5]

```
should_review_skills :=
    skill_nudge_interval > 0
    and iters_since_skill >= skill_nudge_interval
    and skill_manage is available

if final_response and not interrupted:
    spawn_background_review(conversation_snapshot, prompt_variant)
    reset iters_since_skill
```

这里的“interval”是节流机制，不等价于“只要达到 N 次就必然写 skill”。达到间隔只是 fork review 的条件；reviewer 仍然可以判断“nothing to save”。因此正确理解是：计数器负责何时审查，review prompt 和 skill_manage 门禁负责是否真正沉淀。

3.2 审查器：background_review 的信号提取

background_review 的 prompt 明确要求 reviewer 只沉淀稳定、可复用的经验。它优先捕获三类强信号：用户纠正了模型的风格/格式/工作流；任务中出现了非平凡的修复、规避方式、工具模式；本轮加载或查阅的 skill 被证明错误、缺失或过时。[S6]

信号类型	可以沉淀的内容	不该沉淀的内容
用户纠正	“以后分析 PDF 表格必须先截图再下结论”这种跨任务工作流修正。	只针对当前文档的一次性内容。
工具/环境修复	PowerShell 下路径展开、命令参数、渲染验证、缓存坑位等可复现 fix。	已经解决但不具备复用性的瞬时失败。
Skill 缺陷	某个已有 skill 的步骤缺失、默认假设错误、平台条件不完整。	没有证据支持的负面断言，如“工具 X 不可用”。

3.3 写入器：skill_manage 的保护边界

skill_manage 是写入 skill store 的核心工具，动作包括 create、edit、patch、delete、write_file、remove_file。源码里 category 和 name 都有安全字符校验；create 会检查跨目录命名冲突，写入后跑安全扫描；patch 要求模糊匹配唯一，背景写入还要求 read-before-write，失败时回滚；自主后台 review 不能任意修改外部目录、bundled、hub、protected、pinned 等受保护对象。[S7]

门禁	源码含义	解决的风险
name/category 校验	只允许 lowercase、安全字符，category 为单层目录段。	防止路径穿越和混乱目录。
frontmatter 校验	要求 name、description 等字段存在并满足长度/内容约束。	防止不可检索或不可展示的 skill。
name collision 检查	创建时跨本地与外部技能目录查找同名 skill。	避免同名覆盖或检索歧义。
read-before-write	后台 patch 既有 skill 前必须先读过目标。	降低盲改、错改风险。
protected guard	保护 bundled、hub、external、protected、pinned 等。	避免后台自学习破坏系统内置或外部来源。

门禁	源码含义	解决的风险
security scan	写入后扫描，不通过则回滚。	减少 prompt injection、危险脚本、隐私泄露。

3.4 生命周期：usage sidecar 与 Curator

Skill 被创建后并不是永久平铺。skill_usage 通过 .usage.json 记录 created_by、agent_created、view_count、use_count、patch_count、lifecycle、pinned 等元信息。Curator 后台维护会根据活跃度把 skill 标记为 active、stale、archived，并可在开启 consolidation 时合并重复 skill，同时保留报告、备份和 rollback 能力。[S8][S9]

4. 问题一：新增 skill 如何判断所属 category， 以及是否绑定服务/环境

4.1 先区分：category 是目录组织， 不是完整语义分类器

当前源码中，category 主要体现为 skill 的目录路径：`~/hermes/skills/<category>/<name>/SKILL.md`。
skill_manage 对 category 做的是安全性和格式验证，例如小写、单层目录段、不能包含危险路径字符。也就是说，源码层面的 category 判断更像“写到哪个安全目录”，不是一个完整的语义分类模型。[S7]

因此，新增 skill 的 category 不应该只由 LLM 随便起名。工程上应该把它变成一个确定性分类规则：先判定任务类型，再判定主工具/环境，再判定产物类型，最后落到一个稳定 category。服务绑定、运行环境约束和依赖条件不应只靠 category 表达，而应写入 frontmatter 和正文触发条件。

4.2 推荐的 category 判定维度

维度	判定问题	例子	是否应进入 category
任务类别	这个 skill 解决哪类稳定问题？	PDF 表格审查、PowerShell 路径处理、Gmail 整理、论文框架图生成。	是，优先级最高。
主产物	最终交付物是什么？	pdf、docx、slides、spreadsheet、code patch、report。	可作为 category 或 tag。
主工具/服务	必须依赖什么服务或工具？	Gmail、Google Calendar、Docker、Playwright、SQLite、Hermes skill_manage。	通常写入 tags/prerequisites，不建议只靠 category。
运行环境	只在某些环境有效吗？	Windows PowerShell、Linux shell、Docker、browser automation。	写入 platforms/environments。
知识领域	它属于哪类专业知识？	agent-lifecycle、rag、slam、4dgs、latex、pdf。	可作为 category 或 related skill。
权限边界	是否涉及受保护数据或写操作？	公司文档、邮件发送、日历修改、删除文件。	写入正文的安全前置条件，不宜只靠 category。

4.3 “服务绑定”应该怎么表达

如果你的问题里的“绑定”指某个 skill 是否绑定到 cate、gpt、服务、插件或工具，答案是：不要把绑定理解为单一字段。Hermes skill 的适用性是由 metadata、环境过滤、正文触发条件和工具权限共同决定的。源码中的 environments 被解释为 relevance gate，不是硬兼容约束；显式加载 skill 可以绕过 offer-time filter。因此服务绑定必须在多个层面冗余表达，避免误触发。[S10]

绑定层	建议写法	作用
frontmatter.name/description	一句话准确说明何时使用。	参与 skills_list 检索，是第一层召回入口。
platforms	windows、linux、macos、web 等。	避免平台命令误用。
environments	docker、browser、kanban、s6 等 runtime tag。	作为相关性门控；不是绝对安全边界。
prerequisites	需要的 CLI、Python 包、外部文件、登录态。	执行前检查。
required_environment_variables	如 API_KEY 名称，但不要写密钥值。	明确服务依赖并防止凭据泄漏。
metadata.hermes.tags	pdf、gmail、calendar、skill-curation、powershell。	提高多路检索与冲突检测效果。
正文触发条件	Use when / Do not use when / Pitfalls。	防止 category 相近但语义不同的误用。

4.4 分类决策树

```
classify_new_skill(event):
    task_class = extract_reusable_task_class(event)
    artifact = detect_primary_artifact(event)
    tools = detect_required_tools_or_services(event)
    platform = detect_platform_constraints(event)

    if task_class is broad and stable:
        category = canonical_category(task_class, artifact)
    elif tool/service dominates:
        category = canonical_tool_domain(tools[0])
    elif it is only a project-specific reference:
        do not create new category; attach as references/ under existing skill

    write service binding into metadata and body, not only into category
    return category, metadata, trigger_conditions
```

简化成一句话：category 回答“它属于哪类工作流”；metadata 和正文触发条件回答“在哪个服务、哪个环境、哪些前置条件下才应该使用”。

5. 问题二：规模庞大时如何检索 skill 并避免冲突

5.1 Hermes 当前已有的检索路径

Hermes 的基础检索路径是 progressive disclosure：skills_list 先返回技能列表和简短描述，模型根据当前任务选择相关 skill，再通过 skill_view 读取完整 SKILL.md，必要时再打开支持文件。skill_manager 在创建时会跨本地和外部目录查找同名 skill，防止直接 name collision。[S2][S7]

这套机制在 skill 数量较少时足够。但规模变大后，仅靠 name/description 和 LLM 自行判断会出现四类问题：语义重复、触发条件重叠、步骤冲突、陈旧 skill 干扰。因此需要把“检索”和“冲突治理”分开设计。

5.2 大规模 skill 库推荐的五级检索

层级	检索对象	技术方式	过滤/排序信号
L0 精确索引	name、alias、category/name 路径	哈希表或 SQLite unique index	精确命中优先，解决同名和路径定位。
L1 元数据过滤	category、platforms、environments、tags、prerequisites	倒排索引/SQL WHERE	先排除不适配环境和服务的 skill。
L2 词法召回	name、description、Use when、headings	BM25/FTS5	对工具名、错误码、命令名、文件类型敏感。
L3 语义召回	description、触发条件、步骤摘要、pitfalls	embedding + vector index	召回同义表达和跨语言查询。
L4 结构签名匹配	tool signature、artifact signature、input/output pattern	规则 + feature vector	区分“PDF 生成”与“PDF 审查”等相近任务。
L5 LLM rerank/reviewer	Top-K 候选全文或摘要	pairwise/rerank prompt	判断是否真适配、是否重复、是否冲突。

5.3 冲突类型与处理动作

冲突类型	识别信号	处理动作
名称冲突	相同 name 或相同 category/name。	拒绝创建；要求 patch 既有 skill 或改名。
语义重复	description/trigger embedding 高相似，且工具/产物签名相同。	合并到 umbrella；新内容作为 Pitfalls/Procedure/References。
触发条件重叠	两个 skill 都声称处理同类任务，但 Do not use when 不清晰。	补充边界条件；拆成父 skill + 子场景 references。
步骤矛盾	同一环境下对同一动作给出相反命令或顺序。	保留经过验证的新步骤；旧步骤移入 deprecated 或 archive。
服务绑定冲突	声称适用于 Gmail/Calendar/Docker，但缺少对应权限、变量或环境。	增加 prerequisites 和 failure mode；必要时拆分平台版本。
版本陈旧冲突	依赖 API/CLI/论文/库版本已变。	更新 last_verified_at、version note；旧版归档。
安全冲突	包含密钥、越权写操作、跳过审批、危险 shell。	拒绝写入或安全扫描回滚；需要人工确认。

5.4 “避免冲突”的关键不是只做去重，而是写入前预检

合理流程是：先把候选经验抽象成 task_class、tools、artifact、platform、trigger、do_not_use_when；再检索 Top-K 相似 skill；之后做冲突评分。如果相似度高而新增内容只是补丁，就 patch；如果相似度中等但属于同一 umbrella，就写 references 或 templates；只有相似度低且通过可复用性判断，才创建新 skill。

```
retrieve_candidates(event):
```

```

q = build_query(event.task_class, event.tools, event.artifact, event.error_signature)
candidates = exact_lookup(q.name)
            + metadata_filter(q.platform, q.env, q.tags)
            + bm25(q.text)
            + vector_search(q.summary)
reranked = llm_or_cross_encoder_rerank(candidates)
return top_k(reranked, k=10)

conflict_score(candidate, new_skill):
    return 0.35 * semantic_overlap
           + 0.20 * trigger_overlap
           + 0.20 * tool_signature_overlap
           + 0.15 * contradiction_score
           + 0.10 * freshness_or_protection_penalty

```

5.5 推荐索引结构

如果要真正支持大规模 skill 库，建议维护一个派生索引，而不是每次遍历文件系统。索引可以是 SQLite + FTS5 + 向量列/外部向量库，也可以是 JSONL + embedding cache。关键是索引来自文件系统，不替代文件系统；skill_manage 写入成功后原子更新索引，Curator 合并或归档后同步索引。

字段	来源	用途
skill_id	category/name 或内容 hash	稳定标识。
path	文件系统路径	打开和写入定位。
name/description	frontmatter	首轮召回。
category/tags/platforms/environments	frontmatter + 路径	过滤。
trigger_summary	正文 Use when / Do not use when	减少误召回。
tool_signature	正文工具步骤/依赖	服务绑定与冲突检测。
embedding	description + trigger + pitfalls	语义召回。
usage stats	.usage.json	排序和生命周期。
protected/pinned/source	.usage.json + manifest	写入门禁。

6. 问题三：自进化的沉淀标准、评估标准与反复 patch 治理

6.1 什么经验值得沉淀

Hermes 的 background review prompt 已经给出强约束：不要把顺利完成、没有新技术的普通任务都写成 skill；也不要记录一次性故事、环境瞬时失败、负面工具断言。真正值得沉淀的是“未来同类任务可直接复用”的流程、修复、验证清单、坑位和偏好。[S6]

沉淀标准	判断问题	例子
稳定性	这个经验是否不依赖当前一次文件、一次用户临时要求？	“PDF 含图表时先渲染截图再判断”，稳定。
可复用性	未来是否会被相似 query 检索到？	“PowerShell 中 ~ 路径不能直接传给 sqlite3”可复用。
可执行性	是否能转成明确步骤、命令、检查项？	写入命令模板、失败症状、验证命令。
可验证性	是否有明确成功标准？	生成 PDF 后必须 render to PNG 并检查无乱码/裁切。
差异性	是否补充了已有 skill 没有的内容？	已有 PDF skill 没有某个表格截图 pitfall，则 patch。
边界清晰	什么时候不用它？	只适用 Windows PowerShell，不适用 bash。
安全合规	不包含密钥、隐私、不绕过审批。	API key 只写变量名，不写值。

6.2 反面清单：不应该写入的内容

- 只属于当前任务的一次性事实，例如某篇论文某一页的具体结论，除非它是某个论文阅读 skill 的模板样例。
- 没有复现证据的判断，例如“某工具不能用”“某网站一定过期”。这种负面断言很容易过时。
- 纯用户隐私或短期偏好，例如临时说“这次报告用蓝色”。除非用户明确要求长期记住。
- 过窄的 session artifact，例如 PR 编号、一次错误栈、一次文件路径，不能直接变成 skill 名称。
- 已经被已有 skill 覆盖的流程，除非新增内容能作为 Pitfall、Validation 或平台分支。

6.3 评估标准：从“能写”升级到“值得写”

评估层	当前 Hermes 已有机制	建议增强
语法层	frontmatter 必填、name/category 安全字符、size limit。	增加 schema version、owner、last_verified_at、confidence。
检索层	skills_list/skill_view，name collision 检查。	增加 FTS + embedding + service signature，写入前相似 skill 检索。
语义层	background prompt 要求 class-level skill、避免一次性叙事。	增加 novelty score、actionability score、scope score。
执行层	部分安全扫描和回滚。	为有脚本的 skill 自动跑 smoke test；为文档 skill 加 render/QA checklist。
冲突层	创建时同名检测，patch 读前写。	增加语义重复、触发重叠、步骤矛盾检测。
生命周期层	.usage.json + Curator stale/archive/consolidate。	把低使用率且高相似 skill 自动放入 consolidation review queue。

6.4 如何避免 “ 反复贴 skill / 反复 patch ”

反复 patch 的根因通常不是 patch 本身，而是缺少 “ 新增内容归属判断 ”。同一类经验应聚合到 umbrella skill 的固定小节中，而不是每次新建相似 skill。建议把 skill 主体拆成稳定骨架：When to use、Inputs、Procedure、Validation、Pitfalls、Failure modes、Do not use when、References。新增经验只能进入对应槽位，不允许附加流水账。

问题表现	原因	治理方式
每次任务都生成新 skill	缺少相似 skill 检索和 novelty gate。	创建前强制 Top-K 相似检索；高相似则 patch。
同一 skill 被越改越长	把 reference、模板、日志都塞进 SKILL.md。	主文档只留执行骨架；长内容写 references/。
patch 互相覆盖	没有 read-before-write 或 patch 位置不唯一。	后台 patch 必须先 skill_view；patch string 必须唯一；失败则 no-op。
旧规则和新规则矛盾	缺少版本和验证时间。	增加 last_verified_at 与 deprecated note；Curator 归档旧版本。
低质量经验污染检索	审查器遇到 interval 就想写。	允许 no-op；增加 min_quality_score 和人工审核队列。

7. 问题四：进化后如何判断新增 skill 还是改进既有 skill

7.1 源码 prompt 给出的优先级

background_review 的 skill review prompt 对此给出明确偏好顺序：第一，若本轮已经加载了相关 skill，优先更新它；第二，若没有加载但已有合适的 umbrella skill，先通过 skills_list/skill_view 找到并更新；第三，如果只是补充长材料、模板、脚本或样例，则写到已有 umbrella 的 support file；第四，只有当没有已有 skill 覆盖该任务类别时，才创建新的 class-level umbrella skill。[S6]

```
preferred write order:
1. patch currently loaded skill
2. patch existing umbrella skill found via skills_list + skill_view
3. add support file under existing umbrella
4. create a new class-level umbrella skill
5. otherwise do nothing
```

7.2 新增 vs 更新的判定表

场景	判断标准	动作
本轮使用过的 skill 步骤不完整	问题与 loaded skill 的目标一致，只是缺少 pitfall/验证/平台分支。	patch 当前 skill。
已有同类 umbrella skill	Top-K 检索出现同类 skill，trigger 覆盖新场景。	patch umbrella；不要新建。
新增内容很长但不是主流程	API 文档、复杂命令表、模板、长样例。	write_file 到 references/templates/examples/scripts。
新经验只适用某平台	核心任务相同，但 Windows/Linux 命令不同。	在既有 skill 增加 platform branch，不新建，除非步骤完全不同。
新任务类别稳定且库中没有	现有 skill 触发条件不覆盖，且未来可复用。	create class-level umbrella skill。
只是一次 bug 或一次文件	没有跨任务复用价值。	do nothing 或写入当前任务摘要，不进入 skill。
目标 skill 是 hub/bundled/external/protected	后台自主写入受保护。	不自动编辑；可创建本地 override 或提示人工审核，视策略而定。

7.3 “class-level umbrella skill” 的边界

源码 prompt 特别强调，新 skill 的名字必须是 class-level，不是 PR 编号、错误字符串、feature codename 或 session artifact。原因是 skill 要服务未来检索；过窄名称会让库膨胀，过宽名称会导致触发误召回。

不合格名称	问题	合格名称
fix-issue-218-sqlite-path	一次 issue 编号，不能泛化。	windows-sqlite-path-handling
pdf-formula-missing-in-paper-x	绑定单篇论文，未来难检索。	pdf-math-rendering-validation
gpt-cate-binding-bug	语义不清，缩写不可检索。	hermes-skill-category-selection
my-report-blue-style	用户一次性样式。	report-layout-style-guide，前提是多次稳定复用。

7.4 更新既有 skill 的粒度

- 改正文流程：当现有步骤顺序错误、缺少必须验证、默认假设错误时，直接 patch 主体 Procedure。
- 补充坑位：当失败来自边界条件或平台差异时，加入 Pitfalls 或 Troubleshooting。
- 补充适用/不适用条件：当 skill 误触发或误用时，修改 Use when / Do not use when。
- 补充长材料：当内容是参考手册、命令表、模板、脚本时，放到 references/templates/scripts，不膨胀 SKILL.md。

- 拆分 skill：当一个 skill 已经覆盖两个互斥工作流，且检索经常误召回时，拆成 umbrella + 两个子 skill 或 references。

8. 建议的工程化算法：候选检索、相似度评分、冲突评分、写入决策

8.1 从对话中抽取 learning event

background reviewer 不应直接把整段对话压成 skill，而应先抽取 learning event。一个 learning event 至少包含：问题类型、触发症状、修复步骤、适用环境、必需工具、验证方式、是否来自用户纠正、是否本轮加载了相关 skill。

字段	示例	用途
task_class	PDF 生成后公式缺失修复	决定 category 与 umbrella。
trigger	用户指出“公式没有渲染”	决定 Use when。
fix	检索原论文、补全公式、重新排版、渲染验证	决定 Procedure。
tools	web, PDF renderer, ReportLab/LibreOffice	决定服务绑定和前置条件。
platform	container/Linux 或 Windows PowerShell	决定平台分支。
validation	render PDF to PNG, inspect pages	决定质量门禁。
source_skill	本轮加载的 pdf skill	决定优先 patch 谁。

8.2 写入决策评分

可以把新增/更新判断形式化为 scoring。下面阈值是建议，不是 Hermes 当前源码常量。实际系统可以从历史 review 成功率和人工反馈中校准。

```
score_existing(candidate, event):
    return 0.30 * semantic_similarity(candidate.trigger, event.trigger)
        + 0.20 * task_class_match(candidate, event)
        + 0.15 * tool_signature_match(candidate, event)
        + 0.10 * platform_match(candidate, event)
        + 0.10 * loaded_skill_bonus(candidate, event)
        + 0.10 * freshness(candidate)
        + 0.05 * usage_quality(candidate)

if best_score >= 0.78:
    patch best candidate
elif 0.55 <= best_score < 0.78:
    add support file or ask human / no-op
else:
    if novelty_score >= 0.70 and quality_score >= 0.80:
        create new class-level umbrella skill
    else:
        no-op
```

8.3 质量门禁评分

分数	计算依据	低分处理
novelty_score	与已有 skill 的低相似度 + 新任务类别 + 新工具签名。	低则不新建，改为 patch。
actionability_score	是否包含明确步骤、命令、输入输出、验证标准。	低则不写，或只写 memory/notes。
stability_score	是否跨 session、跨文件、跨项目仍成立。	低则 no-op。
safety_score	是否含密钥、越权指令、危险命令。	低则拒绝写入。
conflict_score	触发重叠、步骤矛盾、服务绑定冲突。	高则进入人工 review 或 consolidation。

8.4 最终状态机

```
EXTRACT_SIGNAL
-> RETRIEVE_SIMILAR_SKILLS
    -> NO_REUSABLE_SIGNAL: no-op
    -> EXISTING_MATCH: patch skill
    -> EXISTING_UMBRELLA_LONG_DETAIL: write support file
    -> NOVEL_CLASS: create skill
    -> PROTECTED_TARGET: no-op or manual review
    -> HIGH_CONFLICT: consolidation queue
-> VALIDATE_SCHEMA_AND_SECURITY
-> ATOMIC_WRITE
-> UPDATE_USAGE_SIDEKARE_AND_INDEX
-> CURATOR_LIFECYCLE
```


9. Hermes 现有实现的边界与可增强点

9.1 当前实现已经做得比较稳的部分

- 背景审查与主回答解耦，不阻塞用户当前响应；主对话 prompt cache 不被后台 review 直接污染。
- reviewer 有明确的沉淀信号和反面约束，允许什么都不保存。
- skill_manage 对 name/category/frontmatter/size/安全扫描/保护目录做了基本门禁。
- read-before-write 降低了后台盲目 patch 既有 skill 的风险。
- Curator 通过 usage sidecar 维护 active/stale/archived，并提供 backup/rollback/pin/consolidation。

9.2 仍然需要增强的部分

问题	原因	建议
category 语义弱	源码主要校验目录名，不负责复杂分类。	引入 canonical category registry，写入前进行 category recommendation + validation。
语义冲突检测不足	现有 name collision 不能发现同义重复或步骤矛盾。	增加 embedding 相似度、trigger overlap、tool signature diff。
interval 触发偏机械	达到工具迭代阈值会 fork review，但任务质量信号未必强。	用事件强度 gating：用户纠正、失败修复、skill 被证伪优先触发。
低质量 skill 可能堆积	自动创建如果没有人工/评测门禁，容易“经验垃圾化”。	引入 pending review queue、quality score、最小可执行单元校验。
服务绑定不够显式	environment tag 只是相关性门控，不是硬约束。	在 skill metadata 中标准化 tools、permissions、env vars、platforms、failure modes。
外部 skill 与本地 skill 的覆盖关系复杂	local precedence、external read-only、hub protected 需要解释清楚。	索引展示 source、writability、override path，避免后台写错位置。

9.3 与 hermes-agent-self-evolution 的区别

外部 self-evolution 项目更像 CI/优化器：读取 Hermes 当前 prompt、tool description、skills、benchmarks 和 session DB，生成候选变体，用 GEPA/DSPy/evals 评估，通过门禁后开 PR。它适合做批量优化和回归评估；而 Hermes 主进程的 background_review 更适合在线捕获“刚刚发生的可复用经验”。两者可以互补：在线 review 负责收集经验，离线 self-evolution 负责重构、去重、评测和 PR 化。[S4]

维度	在线 background_review	离线 self-evolution pipeline
输入	当前对话快照、加载过的 skill、用户纠正。	仓库文件、benchmarks、session DB、eval harness。
输出	skill/memory 增量写入或 patch。	候选变体、评测结果、PR。
优势	及时捕获真实失败和偏好。	能批量评测、回归、优化全局一致性。
风险	局部经验质量不稳定。	评测集覆盖不足会过拟合。
适用	小步沉淀、快速修补。	周期性整理、合并、性能优化。

10. 可直接落地的实现清单

10.1 Skill 写入前检查清单

1. 是否有明确的可复用任务类别，而不是一次性项目故事？
2. 是否已经检索过 loaded skill、同 category skill、相似 description skill？
3. 是否应该 patch 既有 skill，而不是新建？
4. 新内容应放在 SKILL.md 主体，还是 references/templates/scripts？
5. frontmatter 是否包含准确 name、description、platforms、environments、tags、prerequisites？
6. 正文是否包含 Use when、Procedure、Validation、Pitfalls、Do not use when？
7. 是否含密钥、隐私、越权写操作、危险命令？
8. 是否能通过 schema 校验、size limit、安全扫描、渲染/运行验证？
9. 是否更新 .usage.json 或派生索引，确保后续可检索和可治理？

10.2 推荐的 SKILL.md 模板

```
---
name: hermes-skill-category-selection
description: Choose whether to create a new Hermes skill or patch an existing one,
  and assign stable category, environment, and service metadata.
platforms: [linux, windows, macos]
environments: [hermes]
prerequisites:
  - Access to skills_list and skill_view before writing.
  - skill_manage available for local writable skills.
metadata:
  hermes.tags: [skill-curation, self-evolution, conflict-detection]
---
```

When to use
Use when a conversation produces a reusable workflow, correction, or pitfall and the agent must decide whether to create a new skill, patch an existing skill, or do nothing.

Procedure

1. Extract reusable task class, tools, artifact, platform, trigger, and validation.
2. Retrieve loaded and related umbrella skills.
3. If a loaded or existing skill covers the task, patch it.
4. If only long reference/template/script content is new, write a support file.
5. Create a new class-level umbrella skill only if no existing skill covers it.
6. Validate frontmatter, safety, and conflict score before writing.

Validation

- Name is class-level, not a session artifact.
- Trigger and do-not-use boundaries are explicit.
- No duplicate or contradictory existing skill remains unresolved.

10.3 一个完整例子：PDF 公式渲染缺失

步骤	判断
事件	用户指出“ PDF 中几处公式没有渲染 ”。
可复用性	这是 PDF 生成/学术文档排版中的稳定坑位，未来会重复出现。
检索	先检索 PDF 生成/文档转换相关 skill。
决策	如果已有 pdf creation skill，patch Validation/Pitfalls；不应新建“ 某篇论文公式缺失 skill ”。
写入内容	增加：生成后必须 render PDF 到 PNG；抽查公式页；LaTeX 公式不要以纯文本残留；必要时从原论文补公式但必须标注来源。
验证	重渲染后页面无缺字、无裁切、公式正常显示。

10.4 一个完整例子：Windows PowerShell 路径错误

步骤	判断
事件	sqlite3 ~/.codex/logs_2.sqlite 在 PowerShell 报 unable to open database。
可复用性	跨任务复用；Windows PowerShell 与 Linux shell 路径展开差异常见。
检索	检索 windows、powershell、sqlite、path-handling 相关 skill。
决策	若已有 windows-powershell-path skill，patch；若没有，可创建 class-level skill。
写入内容	说明 PowerShell 中优先使用 \$env:USERPROFILE\codex\logs_2.sqlite 或 Resolve-Path；不要直接沿用 bash 的 ~ 语义。
验证	命令能找到文件；失败时给出定位目录和权限检查步骤。

11. 证据来源与源码文件索引

以下来源用于支持本文对 Hermes 自进化、skill 系统、写入工具、后台 review 和生命周期治理的分析。网页和源码可能随 upstream main 分支更新；若要复现实验，应固定 commit hash。

编号	来源	本文使用点
S1	Hermes Agent README, GitHub: https://github.com/wiktor-becker/hermes-agent	Hermes 被描述为 self-improving, 会从经验创建/改进 skills, 并与 agentskills 兼容。
S2	Skills System docs: https://wiktor-becker.github.io/hermes-agent/skills/	Skill 目录、progressive disclosure、/learn、skills_list、skill_view。
S3	Hermes Curator docs: https://wiktor-becker.github.io/hermes-agent/curator/	active/stale/archived 生命周期、usage sidecar、consolidation、backup/rollback/pin。
S4	hermes-agent-self-evolution README/PLAN: https://github.com/wiktor-becker/hermes-agent-self-evolution	外部优化管线：DSPy/GEPA、eval gates、PR 化、优化 skill/tool description/system prompt/code。
S5	src/hermes/agent/turn_finalizer.py	skill review interval、iters_since_skill、final response 后台 fork 的触发逻辑。
S6	src/hermes/agent/background_review.py	沉淀信号、反面约束、loaded skill -> existing umbrella -> support file -> new umbrella 的优先级。
S7	src/hermes/tools/skill_manager_tool.py	skill_manage create/patch/write_file/delete ; name/category/frontmatter 校验、collision、read-before-write、protected guard、security scan。
S8	src/hermes/tools/skill_usage.py	.usage.json、created_by、agent_created、view/use/patch_count、lifecycle、pinned。
S9	Curator implementation and docs	定时维护、stale/archive、consolidation、reports、rollback。
S10	src/hermes/agent/skill_utils.py and skills_tool.py	environments 是 relevance gate, 不是硬兼容；skills 是目录化 SKILL.md + 支持文件。

11.1 结论压缩版

- 新增 skill 的 category：按任务类别/产物/工具环境确定目录，但服务绑定应写在 metadata 与正文触发条件里。
- 检索与冲突：用 exact + metadata + BM25 + embedding + tool signature + LLM rerank；写入前做 duplicate/overlap/contradiction/protected 检查。
- 沉淀标准：只有稳定、可复用、可执行、可验证、非一次性的经验才沉淀；普通顺利任务 no-op。
- 新增还是更新：源码偏好顺序是 loaded skill、existing umbrella、support file、新 class-level umbrella。
- Curator 负责后续生命周期，避免 skill pile-up；外部 self-evolution 负责更重的 eval/PR 化优化。